

ENGENHEIRO DE QUALIDADE DE SOFTWARE

Eduardo de Araujo Soares

Análise de Qualidade

São Paulo
2026

1. RESUMO

Este trabalho apresenta a estratégia de qualidade e a automação de testes aplicadas ao e-commerce EBAC-SHOP, conduzidas como parte de um time ágil. Foram analisadas oito histórias de usuário (carrinho, login, API de cupons, catálogo de produtos, painel Minha Conta, meus pedidos, endereços e detalhes da conta), para as quais foram escritos critérios de aceitação em Gherkin e 49 casos de teste, aplicando partição de equivalência, análise de valor limite, tabela de decisão e transição de estado.

A automação cobre a interface web com Cypress e Playwright, usando Page Object, a API de cupons com Supertest, Jest e validação de contrato com Ajv, e a performance com K6 (20 usuários virtuais, 2 minutos e rampup de 20 segundos). Os relatórios são gerados com Mochawesome, Allure e jest-html-reporters.

A execução é automatizada com GitHub Actions: um workflow manual executa em sequência os testes de UI (Playwright e Cypress), de API (Supertest e Pytest) e de performance (K6) e publica os relatórios como artefatos de cada execução.

2. SUMÁRIO

1. RESUMO.....
2. SUMÁRIO.....
3. INTRODUÇÃO.....
4. O PROJETO.....
4.1 Estratégia de teste.....
4.2 Critérios de aceitação.....
4.2.1 Histórias refinadas.....
4.2.2 Histórias criadas para este trabalho.....
4.3 Casos de testes.....
4.4 Repositório no Github.....
4.5 Testes automatizados.....
4.5.1 Automação de UI.....
4.5.2 Automação de API.....
4.6 Integração contínua.....
4.7 Testes de performance.....
5. CONCLUSÃO.....
6. REFERÊNCIAS BIBLIOGRÁFICAS.....

3. INTRODUÇÃO

A EBAC-SHOP é uma loja virtual de testes usada neste trabalho como sistema sob teste. O objetivo é aplicar, de ponta a ponta, o fluxo de trabalho de um Quality Engineer: entender as histórias de usuário, definir critérios de aceitação, projetar casos de teste, automatizar os de maior valor e medir o comportamento da aplicação sob carga.

O trabalho parte de três histórias já refinadas (US-0001 – Adicionar item ao carrinho, US-0002 – Login na plataforma e US-0003 – API de cupons) e as complementa com cinco histórias escritas para as funcionalidades Catálogo de Produtos, Painel Minha Conta, Meus Pedidos, Endereços e Detalhes da Conta, nomeadas US001 a US005.

Os capítulos seguintes descrevem a estratégia de testes, os critérios de aceitação em Gherkin, os casos de teste, o repositório, as automações de UI e API, a integração contínua e os testes de performance. Espera-se, como resultado, um repositório organizado e reproduzível, que sirva de base para a evolução contínua da cobertura de testes.

4. O PROJETO

Os testes foram desenvolvidos para validar o e-commerce EBAC-SHOP (http://lojaebac.ebaconline.art.br/), considerando as histórias de usuário como se o trabalho fosse realizado dentro de um time ágil. As etapas a seguir acompanham o fluxo do planejamento à entrega.

4.1 Estratégia de teste

A estratégia foi organizada em ramos resumidos na tabela a seguir.

Diretriz	Definição adotada
Objetivo	Garantir que as funcionalidades de compra e conta do cliente atendam às regras de negócio das histórias, detectando falhas cedo e de forma repetível.
Papéis e responsabilidades	Quality Engineer: planeja, escreve critérios e casos, automatiza, executa e reporta. Solicitante/Product Owner: define as regras de negócio. Desenvolvimento: corrige os defeitos encontrados.
Fases de teste	Análise das histórias, critérios de aceitação, projeto de casos, automação, execução, relatório e integração contínua.
Tipos de teste	Funcional, de regressão, de contrato (API), de performance (carga, pico e estresse) e exploratório manual.
Técnicas	Partição de equivalência, análise de valor limite, tabela de decisão, transição de estado e teste de caso de uso.
Padrões	Gherkin (BDD) para critérios, Page Object e camada de fluxo (Flow) na UI, camada de serviço na API e dados de teste separados.
Abordagem	Automatizada nos casos de maior valor (caminho feliz e alternativo); manual nos demais, com rastreabilidade na planilha de casos.
Plataformas	Web e API.
Ferramentas	Cypress (JavaScript), Playwright (Python e Pytest), Supertest com Jest, Ajv, K6, Mochawesome, Allure, jest-html-reporters, GitHub e GitHub Actions.
Ambientes	Loja pública EBAC-SHOP (http://lojaebac.ebaconline.art.br/)

4.2 Critérios de aceitação

As histórias “Adicionar item ao carrinho”, “Login na plataforma” e “API de cupons” possuem critérios de aceitação em linguagem Gherkin (mais de 4 cenários cada), disponíveis na pasta Gherkin do repositório. As cinco funcionalidades restantes foram descritas como histórias de usuário, com regras de negócio (RN) e critérios em Gherkin, apresentados em seguida.

4.2.1 Histórias refinadas

US-0001 – Adicionar item ao carrinho (carrinho.feature)

#	Cenário
1	Remover item do carrinho
2	Seguir para pagamento e retornar
3	Não permitir adicionar item indisponível ao carrinho
4	Adicionar itens ao carrinho
5	Adição de produtos com variações no carrinho
6	Aplicação de desconto conforme valor do carrinho
7	Limite de itens do mesmo produto no carrinho
8	Limite de valor total do carrinho

US-0002 – Login na plataforma (login.feature)

#	Cenário
1	Autenticação com credenciais válidas
2	Bloqueio de acesso após multiplas tentativas
3	Não permitir login de usuário inativo
4	Manter o bloqueio durante os 15 minutos
5	Liberar o login após o período de bloqueio
6	Autenticação com credenciais invalidas

US-0003 – API de cupons (apiCupons.feature)

#	Cenário
1	Acesso não autorizado à API de cupons
2	Cadastrar cupom com sucesso
3	Validação de erro ao cadastrar cupom
4	Não permitir cadastro de cupons com código repetido
5	Listar todos cupons
6	Listar cupom com ID válido
7	Listar cupom com ID inválido
8	Editar cupom com ID válido
9	Editar cupom com ID inválido

4.2.2 Histórias criadas para este trabalho

US001 – Catálogo de Produtos

Projeto	EBAC-SHOP	Versão	001
Tipo	Feature	Story point	7

Solicitante	Eduardo de Araujo	Prioridade	Alta

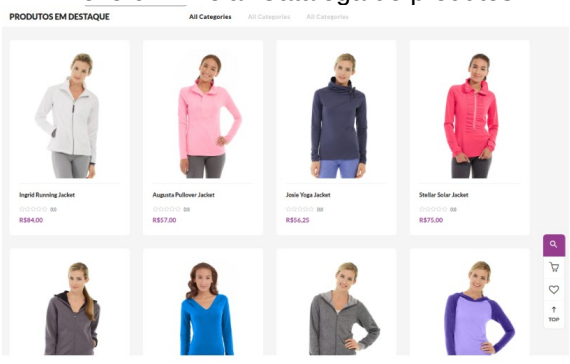
Descrição
Como cliente da EBAC-SHOP Quero visualizar um catálogo de produtos Para poder escolher um ou mais itens e seguir para o pagamento

ID	Regras de negócio
RN01	Cada item do catálogo deve possuir uma imagem, um nome, um valor e uma avaliação
RN02	Na visualização em grade, ao posicionar o mouse em cima de um produto, opções para, salvar, comparar, ver opções e ver rapidamente devem ser exibidas e permitirem interação.
RN03	Na visualização em lista as opções para, salvar, comparar, ver opções e ver rapidamente devem ser exibidas e permitirem interação.
RN04	Deve ser possível ordenar os produtos por popularidade, média de classificação, mais recente, menor e maior preço.
RN05	Deve ser possível alternar entre visualização dos produtos em lista ou grade

Critérios de aceitação em Gherkin (catalogoProdutos.feature):

#	Cenário
1	Exibir imagem, nome, valor e avaliação em cada item
2	Exibir opções ao posicionar o mouse na visualização em grade
3	Exibir opções na visualização em lista
4	Ordenar os produtos
5	Alternar entre visualização em grade e em lista

Anexo 01 – Tela “Cataloga de produtos”



Fonte: Autoral

US002 – Painel Minha Conta

Projeto	EBAC-SHOP	Versão	001
Tipo	Feature	Story point	5
Solicitante	Eduardo de Araujo	Prioridade	Média

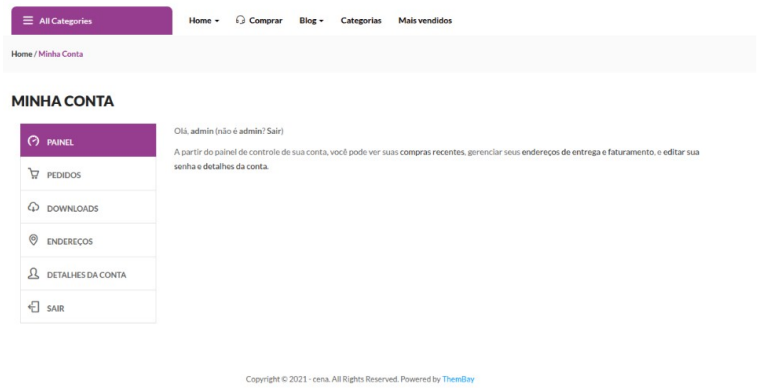
Descrição
Como cliente da EBAC-SHOP Quero possuir um tela reservada para mim quando realizar o cadastro Para ter controle de dados e pedidos referentes ao meu usuário

ID	Regras de negócio
RN01	Deve possuir uma mensagem de boas vindas
RN02	Deve permitir que o usuário faça logout da sua conta
RN03	Só deve estar disponível para usuários com login ativo
RN04	Caso o usuários não possua login ativo, uma tela para login ou registro deve ser carregada no lugar

Critérios de aceitação em Gherkin (painelMinhaConta.feature):

#	Cenário
1	Exibir mensagem de boas-vindas
2	Realizar logout da conta
3	Disponibilizar o painel somente com login ativo
4	Exibir tela de login ou registro sem login ativo
5	Bloquear o painel após o logout

Anexo 02 – Tela “Minha Conta”



Fonte: Autoral

US003 – Meus Pedidos

Projeto	EBAC-SHOP	Versão	001
Tipo	Feature	Story point	6
Solicitante	Eduardo de Araujo	Prioridade	Alta

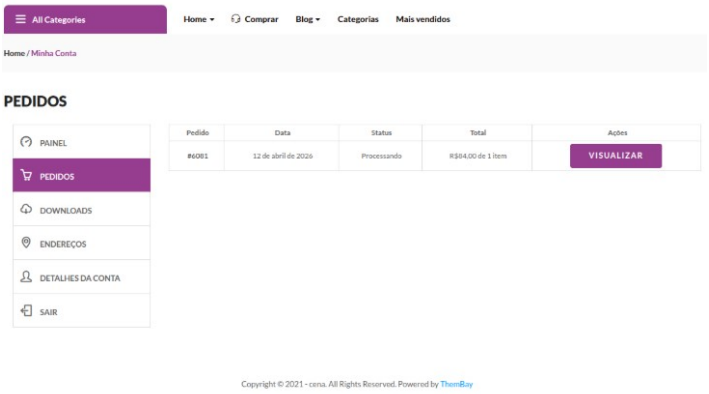
Descrição
Como cliente da EBAC-SHOP Quero ter uma tela dedicada aos pedidos realizados Para validar e acompanhar todos os dados dos itens solicitados pelo meu usuário

ID	Regras de negócio
RN01	Cada produto pedido pago pelo cliente deve aparecer nessa página
RN02	Cada produto deve possuir as seguintes descrições condizentes: Pedido, Data, Status e Total
RN03	Cada produto deve possuir uma opção de “ação”, permitindo ao usuário visualizar com mais detalhes as informações da compra
RN04	Ao clicar na opção ação, uma nova tela deve ser carregada com as seguintes informações: Pedido, Data, Status, Nome do produto, Valor de cada produto, Valor total, Método de pagamento e Detalhes do endereço

Critérios de aceitação em Gherkin (meusPedidos.feature):

#	Cenário
1	Exibir pedidos pagos
2	Exibir Pedido, Data, Status e Total de cada pedido
3	Disponibilizar a ação de visualizar em cada pedido
4	Exibir os detalhes completos do pedido
5	Cliente sem pedidos

Anexo 03 – Tela “Meus pedidos”



Fonte: Autoral

US004 – Endereços

Projeto	EBAC-SHOP	Versão	001
Tipo	Feature	Story point	4
Solicitante	Eduardo de Araujo	Prioridade	Baixa

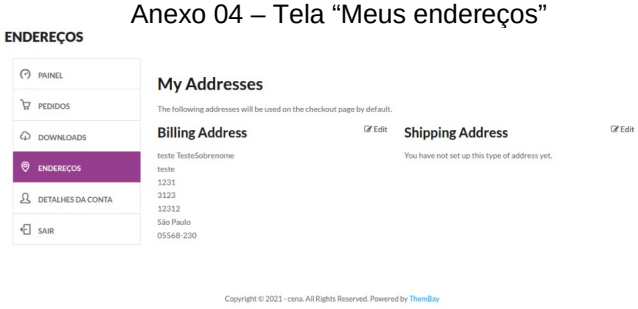
Descrição
Como cliente da EBAC-SHOP Quero ter uma tela dedicada aos meus endereços Para editar e salvar endereços para utilização em pedidos futuros

ID	Regras de negócio
RN01	Todos os campos marcados com (*) são de preenchimento obrigatório. Ao tentar salvar o formulário sem preencher qualquer um desses campos, o sistema deve impedir a gravação e exibir uma mensagem de validação no formato: “[Nome do Campo] é um campo obrigatório.”
RN02	Os campos Nome, Sobrenome, País, Endereço, Cidade, Estado, CEP, Telefone e Endereço de e-mail devem ser obrigatórios
RN03	O campo “Nome da empresa” deve ser opcional
RN04	O modelo de campos para endereço de faturamento deve ser idêntico ao modelo de campos para endereço de entrega
RN05	O usuário pode possuir dois endereços, sendo eles, um de faturamento e outro de entrega, ambos devem ser editáveis e não obrigatórios para a conta, apenas para os pedidos.
RN06	Alterar um endereço não deve mudar o endereço registrado em uma compra anterior com status finalizado ou pendente.

Critérios de aceitação em Gherkin (enderecos.feature):

#	Cenário
1	Validar campos obrigatórios do endereço
2	Salvar endereço sem o nome da empresa
3	Manter o mesmo modelo de campos entre faturamento e entrega
4	Editar cada endereço de forma independente
5	Conta sem endereços cadastrados
6	Preservar o endereço de pedidos anteriores

Anexo 04 – Tela “Meus endereços”



Fonte: Autoral

US005 – Detalhes da conta

Projeto	EBAC-SHOP	Versão	001
Tipo	Feature	Story point	4
Solicitante	Eduardo de Araujo	Prioridade	Baixa

Descrição
Como cliente da EBAC-SHOP Quero ter uma tela dedicada aos detalhes da conta Para editar e salvar dados atrelados ao meu usuário

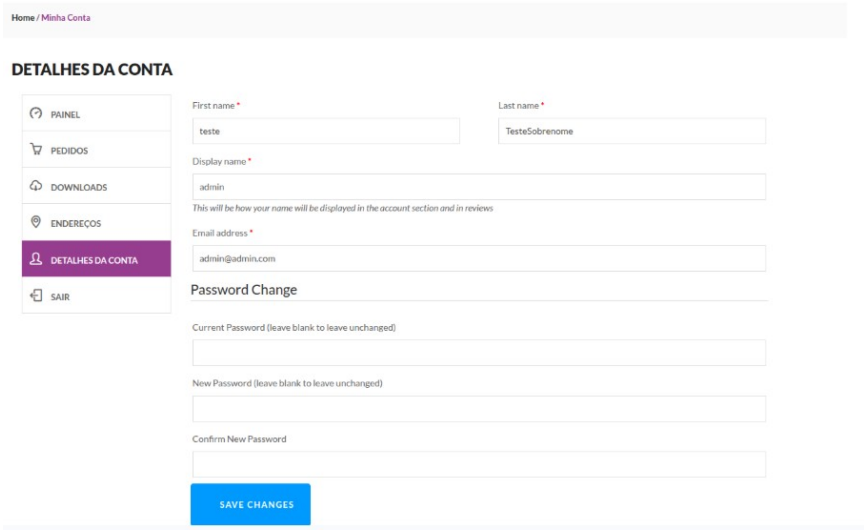
ID	Regras de negócio
RN01	Os campos marcados com (*) são obrigatórios e devem ser validados ao salvar. Ao clicar em “Save changes” com algum desses campos vazio, o sistema deve impedir o salvamento e exibir a mensagem: “[Nome do Campo] é um campo obrigatório.”
RN02	Os campos First name, Last name, Display name, Email address devem ser obrigatórios
RN03	Os campos da seção Password Change são opcionais e só devem ser considerados se pelo menos um deles for preenchido.
RN04	Para continuar, todos os campos da seção Password Change devem ser preenchidos, do contrário, o sistema deve impedir a ação e exibir a mensagem: “Preencha todos os campos de senha.”
RN05	O campo Email address deve conter ao menos um @, do contrário, uma mensagem de alerta deve ser exibida informando que o e-mail fornecido é invalido

Critérios de aceitação em Gherkin (detalhesConta.feature):

#	Cenário
1	Validar campos obrigatórios
2	Atualizar informações pessoais com dados válidos
3	Salvar sem preencher a seção de alteração de senha
4	Alterar a senha com todos os campos preenchidos
5	Exigir todos os campos de senha quando algum for preenchido

#	Cenário
6	Validar o formato do e-mail

Anexo 05 – Tela “Detalhes da conta”



Fonte: Autoral

4.3 Casos de testes

Foram elaborados 49 casos de teste para as oito histórias (de 5 a 8 por história), cobrindo o caminho feliz, os fluxos alternativos e os cenários negativos. Cada caso informa a técnica aplicada, a prioridade, a regra de negócio validada e se foi automatizado. A relação completa, com pré-condições, passos, dados e resultado esperado.

Resumo dos casos por história

História	Funcionalidade	Casos	Auto.	Manual	Feliz	Alt.	Neg.
US-0001	Adicionar item ao carrinho	6	3	3	2	2	2
US-0002	Login na plataforma	6	4	2	1	0	5
US-0003	API de cupons	8	8	0	2	1	5
US001	Catálogo de produtos	6	0	6	2	3	1
US002	Painel Minha Conta	5	0	5	2	1	2
US003	Meus pedidos	6	0	6	2	2	2
US004	Endereços	6	4	2	2	2	2
US005	Detalhes da conta	6	6	0	2	1	3
Total		49	25	24	15	12	22

Relação dos casos de teste

ID	Título	Regra	Fluxo	Técnica	Auto.
CT-US0001-01	Adicionar um produto ao carrinho	-	Feliz	Teste de caso de uso	Sim
CT-US0001-02	Adicionar vários produtos diferentes ao carrinho	-	Feliz	Teste de caso de uso	Sim
CT-US0001-03	Remover item e restaurar pelo link 'Desfazer'	-	Alt.	Transição de estado	Sim
CT-US0001-04	Limite de itens do mesmo produto (10)	Máx. 10 itens do mesmo produto	Neg.	Análise de valor limite	Não
CT-US0001-05	Limite de valor total do carrinho (R$ 990,00)	Valor máx. R$ 990,00	Neg.	Análise de valor limite	Não
CT-US0001-06	Desconto conforme faixa de valor do carrinho	R$ 200 a R$ 600: 10%; acima de R$ 600: 15%	Alt.	Partição de equivalência + valor limite	Não
CT-US0002-01	Login com credenciais válidas	-	Feliz	Teste de caso de uso	Sim
CT-US0002-02	Login com senha incorreta	Mensagem de erro para login/senha errados	Neg.	Tabela de decisão	Sim
CT-US0002-03	Login com usuário inexistente	Mensagem de erro para login/senha errados	Neg.	Tabela de decisão	Sim
CT-US0002-04	Login com campos obrigatórios vazios	-	Neg.	Partição de equivalência	Sim
CT-US0002-05	Bloqueio após 3 senhas incorretas	3 erros de senha: bloqueio de 15 min	Neg.	Análise de valor limite	Não
CT-US0002-06	Login de usuário inativo	Somente usuários ativos fazem login	Neg.	Partição de equivalência	Não
CT-US0003-01	Cadastrar cupom com todos os campos válidos	POST com campos obrigatórios	Feliz	Teste de caso de uso	Sim
CT-US0003-02	Listar todos os cupons	GET lista todos os cupons	Feliz	Teste de caso de uso	Sim
CT-US0003-03	Buscar cupom por ID válido	GET busca por ID	Alt.	Partição de equivalência	Sim
CT-US0003-04	Buscar cupom por ID inexistente	-	Neg.	Partição de equivalência	Sim
CT-US0003-05	Cadastrar cupom sem código	-	Neg.	Partição de equivalência	Sim
CT-US0003-06	Cadastrar cupom com valor negativo	-	Neg.	Análise de valor limite	Sim
CT-US0003-07	Cadastrar cupom com código repetido	Nome do cupom não pode ser repetido	Neg.	Partição de equivalência	Sim
CT-US0003-08	Acessar a API sem autenticação	-	Neg.	Tabela de decisão	Sim
CT-US001-01	Exibir imagem, nome, valor e avaliação de cada item	RN01	Feliz	Teste de caso de uso	Não
CT-US001-02	Opções ao passar o mouse na visualização em grade	RN02	Feliz	Teste de caso de uso	Não
CT-US001-03	Opções na visualização em lista	RN03	Alt.	Teste de caso de uso	Não
CT-US001-04	Ordenação dos produtos	RN04	Alt.	Partição de equivalência	Não

ID	Título	Regra	Fluxo	Técnica	Auto.
CT-US001-05	Alternar entre visualização em grade e em lista	RN05	Alt.	Transição de estado	Não
CT-US001-06	Ordenação com um único produto na lista	RN04	Neg.	Análise de valor limite	Não
CT-US002-01	Exibir mensagem de boas-vindas	RN01	Feliz	Teste de caso de uso	Não
CT-US002-02	Realizar logout	RN02	Feliz	Transição de estado	Não
CT-US002-03	Painel disponível com login ativo	RN03	Alt.	Partição de equivalência	Não
CT-US002-04	Acessar o painel sem login ativo	RN04	Neg.	Partição de equivalência	Não
CT-US002-05	Voltar ao painel após o logout pelo histórico do navegador	RN03 / RN04	Neg.	Transição de estado	Não
CT-US003-01	Pedido pago aparece na lista	RN01	Feliz	Teste de caso de uso	Não
CT-US003-02	Colunas Pedido, Data, Status e Total	RN02	Feliz	Teste de caso de uso	Não
CT-US003-03	Ação de visualizar disponível em cada pedido	RN03	Alt.	Teste de caso de uso	Não
CT-US003-04	Detalhes completos do pedido	RN04	Alt.	Teste de caso de uso	Não
CT-US003-05	Cliente sem pedidos	RN01	Neg.	Partição de equivalência	Não
CT-US003-06	Acessar pedido de outro cliente	RN04	Neg.	Partição de equivalência	Não
CT-US004-01	Atualizar endereço de entrega com dados válidos	RN05	Feliz	Teste de caso de uso	Sim
CT-US004-02	Atualizar endereço de faturamento com dados válidos	RN05	Feliz	Teste de caso de uso	Sim
CT-US004-03	Campos obrigatórios em branco	RN01 / RN02	Neg.	Tabela de decisão	Sim
CT-US004-04	Salvar sem o nome da empresa (campo opcional)	RN03	Alt.	Partição de equivalência	Sim
CT-US004-05	Modelo de campos idêntico entre faturamento e entrega	RN04	Alt.	Teste de caso de uso	Não
CT-US004-06	Alterar endereço não muda o de pedido anterior	RN06	Neg.	Teste de caso de uso	Não
CT-US005-01	Atualizar informações pessoais com dados válidos	RN01	Feliz	Teste de caso de uso	Sim
CT-US005-02	Alterar senha com todos os campos preenchidos	RN03	Feliz	Teste de caso de uso	Sim
CT-US005-03	Campos obrigatórios em branco	RN01 / RN02	Neg.	Tabela de decisão	Sim
CT-US005-04	Senha: preencher apenas alguns campos da seção	RN04	Neg.	Tabela de decisão	Sim
CT-US005-05	E-mail sem arroba	RN05	Neg.	Partição de equivalência	Sim
CT-US005-06	Salvar sem preencher a seção de senha (opcional)	RN03	Alt.	Partição de equivalência	Sim

Os casos automatizados incluem, para cada história com automação, ao menos um caminho feliz e um alternativo ou negativo.

4.4 Repositório no Github

O código-fonte das automações, a documentação em Gherkin e este documento estão versionados no repositório público abaixo.

Link do repositório: https://github.com/Borboletas-Edu/Projeto-completo---EBAC---QE

Pasta do repositório	Conteúdo
Automação - UI - Cypress	Testes de interface com Cypress (pasta de UI)
Automação - UI - Playwright	Testes de interface com Playwright e Pytest (pasta de UI)
Automação - API - Supertest	Testes de API de cupons com Supertest, Jest e validação de contrato
Automação - API - Pytest	Testes de API de cupons com Pytest e Allure
K6 - Testes de Performance	Testes de carga, pico e estresse com K6
Gherkin	Critérios de aceitação das 8 histórias
Casos de Teste	Planilha com os 49 casos de teste

4.5 Testes automatizados

4.5.1 Automação de UI

Para a interface web foram construídos dois projetos equivalentes, um em Cypress (JavaScript) e outro em Playwright (Python com Pytest), cobrindo cadastro, login, carrinho, endereços e detalhes da conta. A tabela compara as ferramentas consideradas.

Critério	Cypress	Playwright	Selenium WebDriver
Linguagens	JavaScript / TypeScript	JavaScript, TypeScript, Python, Java e .NET	Java, Python, JavaScript, C# e outras
Espera por elementos	Automática (retry embutido)	Automática (auto-wait)	Manual (waits explícitos)
Navegadores	Chrome, Edge, Firefox e WebKit (experimental)	Chromium, Firefox e WebKit	Todos, via drivers
Instalação e configuração	Simples (um pacote npm)	Simples (pacote + instalação dos navegadores)	Mais trabalhosa (drivers e bindings)
Execução em paralelo	Nativa em nuvem; local com plugins	Nativa	Via Grid ou frameworks
Depuração	Test Runner visual, time travel e screenshots	Trace Viewer, vídeo e screenshots	Depende da IDE e de plugins
Relatórios	Mochawesome e Allure	Allure, HTML e JUnit	Allure e outros, via framework de teste
Curva de aprendizado	Baixa	Baixa a média	Média a alta

A utilização de Cypress e Playwright em conjunto teve como objetivo conhecer, explorar e comparar ambas as soluções, proporcionando experiência prática com

suas funcionalidades e diferentes abordagens. O Playwright com Python também permitiu aproveitar o conhecimento já existente nas automações de API com Pytest.

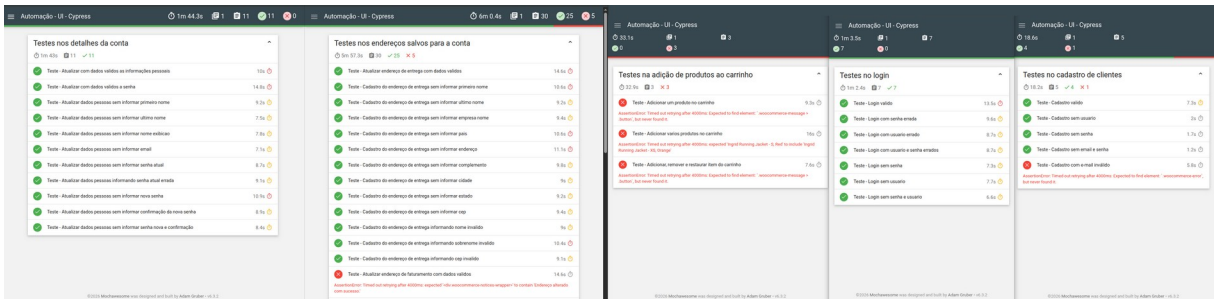
Embora ambas tenham apresentado pontos positivos, considerando a facilidade de trabalho, produtividade, funcionalidades, acompanhamento nativo, integração com a nuvem e experiência adquirida, minha preferência seria pelo **Cypress para dar continuidade ao projeto**, respeitando também os critérios e as necessidades da equipe.

4.5.2 Testing Pattern.

Foram aplicados o Page Object, que centraliza seletores e ações de cada página, e uma camada de fluxo (Flow) que combina ações de páginas em operações de negócio, como realizar login ou editar endereço. Os dados ficam separados dos testes (data-driven), permitindo executar a mesma validação para vários cenários de entrada.

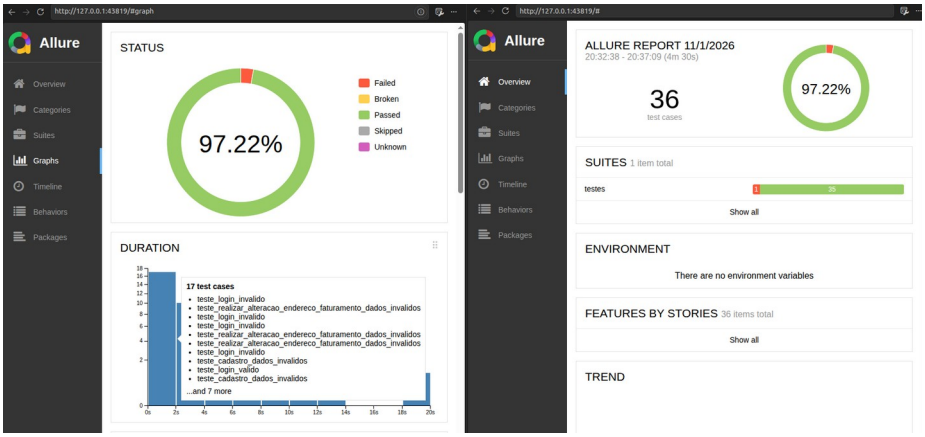
Relatórios. O Cypress gera relatório com Mochawesome e o Playwright com Allure.

Anexo 06 – Relatório Cypress



Fonte: Autoral

Anexo 07 – Relatório Playwright com Pytest



Fonte: Autoral

4.5.2 Automação de API

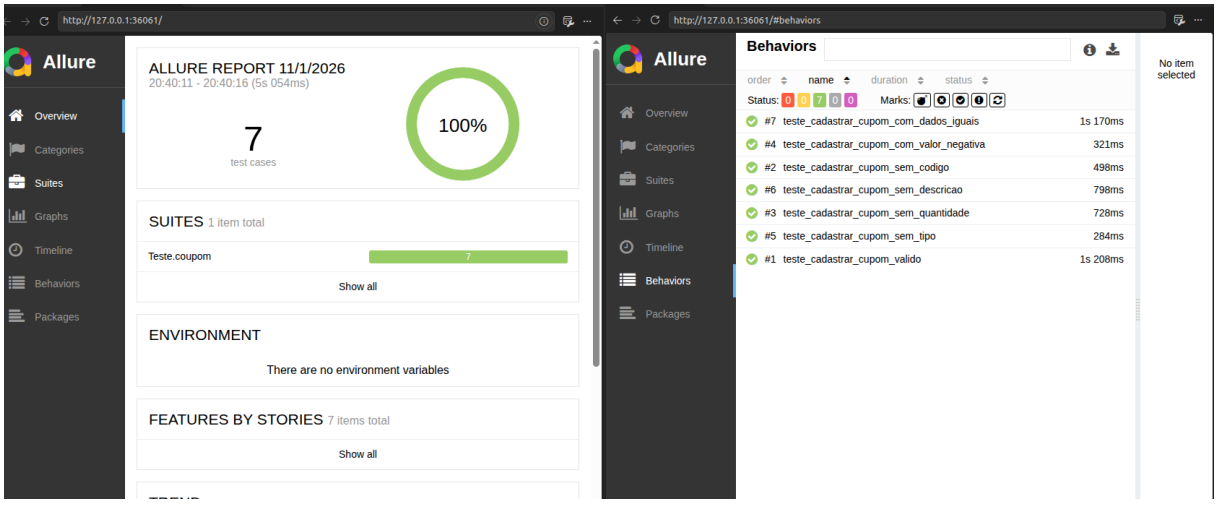
A API de cupons do WooCommerce (/wc/v3/coupons) foi automatizada com Supertest e Jest, no projeto “Automação - API - Supertest”, com autenticação Basic e credenciais fora do código, em arquivo .env. Os 12 testes cobrem:

- GET: listagem de cupons, busca por ID válido, ID inexistente (404) e acesso sem autenticação (401);
- POST: cadastro válido, sem descrição (campo opcional), sem código, valor negativo, sem tipo de desconto, tipo inexistente, código repetido e sem autenticação.

Contrato. As respostas de sucesso (item e lista) e de erro são validadas com Ajv contra schemas versionados em contracts/ (cupomSchema e erroSchema), verificando campos obrigatórios, tipos e valores permitidos. Se o contrato mudar, o teste falha indicando o campo e o motivo.

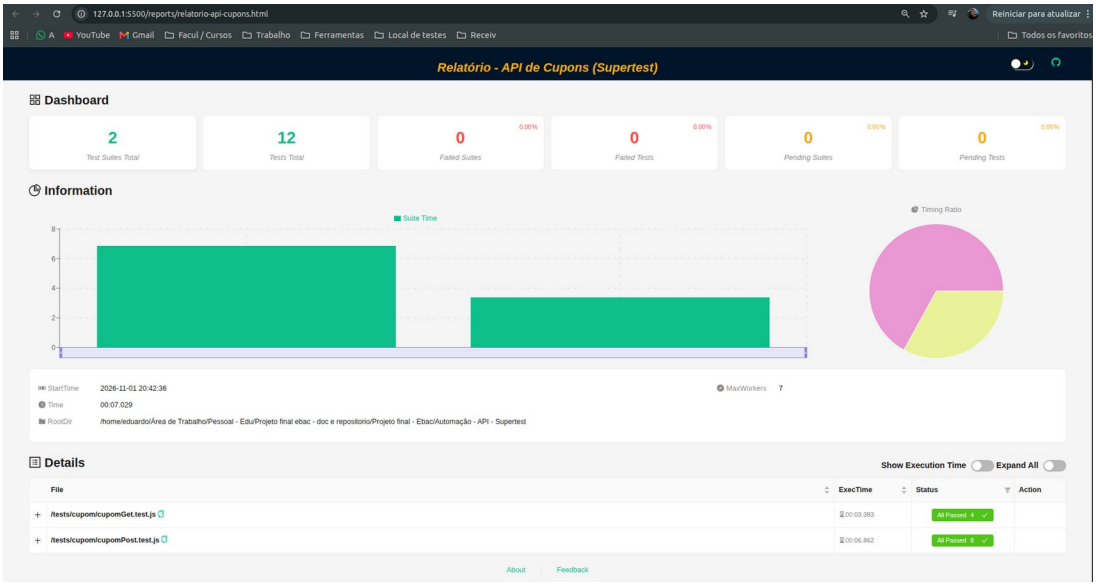
Padrão e relatório. Foi usada uma camada de serviço (utils/cupomApi.js) para as chamadas HTTP, payloads gerados dinamicamente para evitar colisão de códigos e relatório HTML gerado com jest-html-reporters. Existe ainda uma automação equivalente em Pytest, com Allure, na pasta “Automação - API - Pytest”.

Anexo 08 – Relatório API com Pytest



Fonte: Autoral

Anexo 09 – Relatório API com Supertest



Fonte: Autoral

Boas práticas adotadas

- Separação entre dados, páginas, fluxos e especificações nos testes de UI;
- Credenciais em variáveis de ambiente, com arquivo .env.exemple e .env fora do versionamento;
- Dados dinâmicos para evitar conflitos entre execuções;
- Relatórios gerados a cada execução e ignorados no versionamento;
- README em cada projeto explicando estrutura, configuração e execução.

4.6 Integração contínua

A integração contínua foi implementada com GitHub Actions no workflow “QA - Testes automatizados”, versionado em .github/workflows/qa.yml. O disparo é manual (workflow_dispatch), pela aba Actions do repositório, o que permite executar a suíte sob demanda sem sobrecarregar o ambiente público da loja. O workflow usa permissão somente de leitura do conteúdo (contents: read), Python 3.12 e Node.js 22 (LTS).

Os jobs rodam em sequência e cada um é executado mesmo que o anterior tenha falhado, garantindo que todos os relatórios sejam gerados:

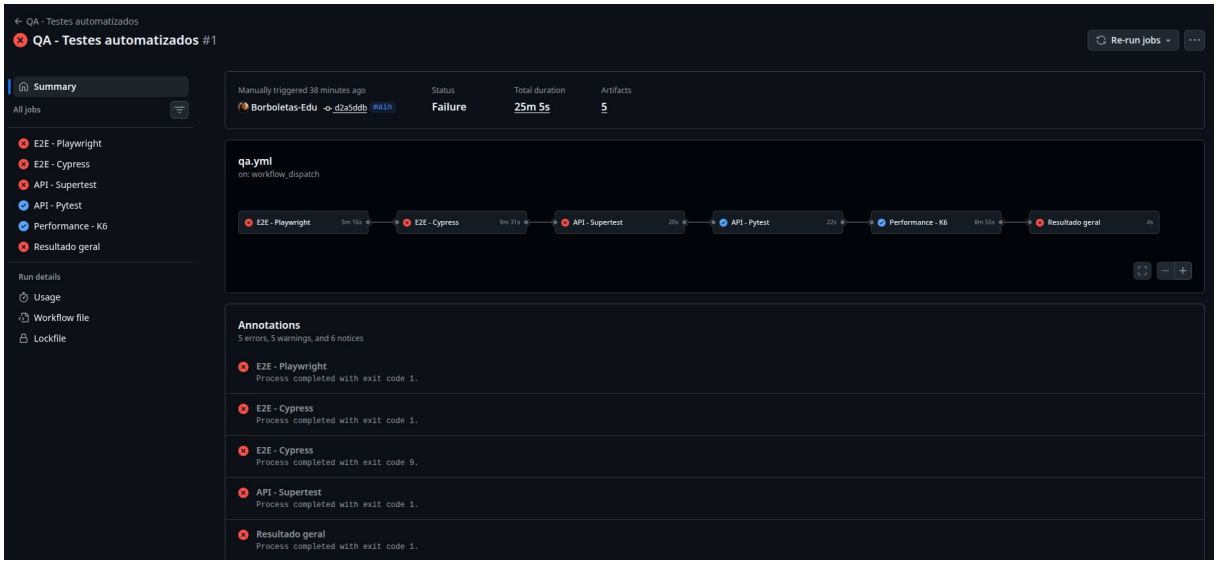
- **E2E – Playwright:** instala as dependências e o navegador Chromium e executa a suíte com Pytest, publicando os resultados do Allure;

- **E2E – Cypress:** instala com npm ci, executa no Chrome, mescla os resultados e gera o relatório HTML do Mochawesome, publicando relatórios e screenshots;
- **API – Supertest:** executa os testes de cupons com validação de contrato e publica o relatório HTML do Jest;
- **API – Pytest:** executa a suíte de cupons e publica os resultados do Allure;
- **Performance – K6:** instala o K6 (grafana/setup-k6-action) e executa os testes de carga, pico e estresse de cadastro e login, exportando um dashboard HTML de cada execução;
- **Resultado geral:** consolida o resultado dos cinco jobs e marca o workflow como falho se qualquer um deles não terminar com sucesso.

Os dados sensíveis (endereço da loja e da API, usuários e senhas da interface, da API e do K6) ficam em GitHub Secrets e são injetados como variáveis de ambiente, sem aparecer no código. As dependências de pip e npm são mantidas em cache para reduzir o tempo de execução. Cada job possui limite de tempo (timeout) de 20 a 30 minutos.

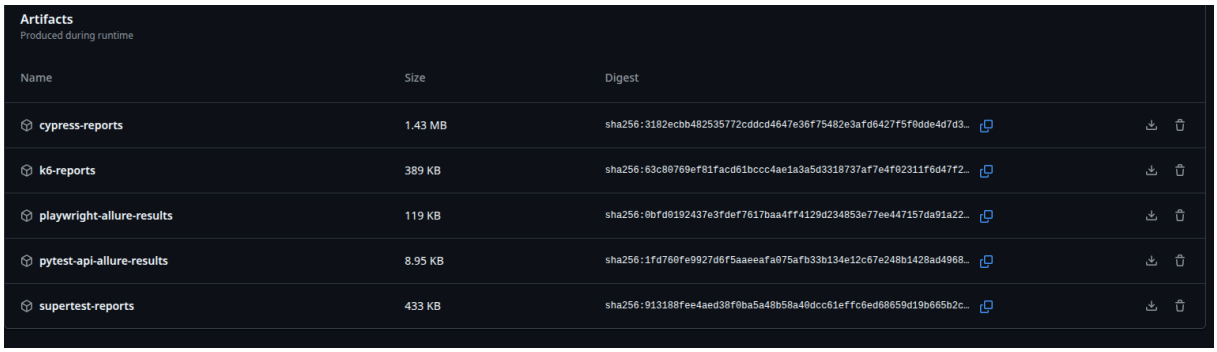
Ao final de cada execução, os relatórios ficam disponíveis como artefatos do workflow (Anexo 11) e o resultado do pipeline executado manualmente é apresentado no Anexo 10.

Anexo 10 – Integração continua executada manualmente



Autor: Autoral

Anexo 11 – Artifacts gerados – Relatórios e screenshots



Autor: Autoral

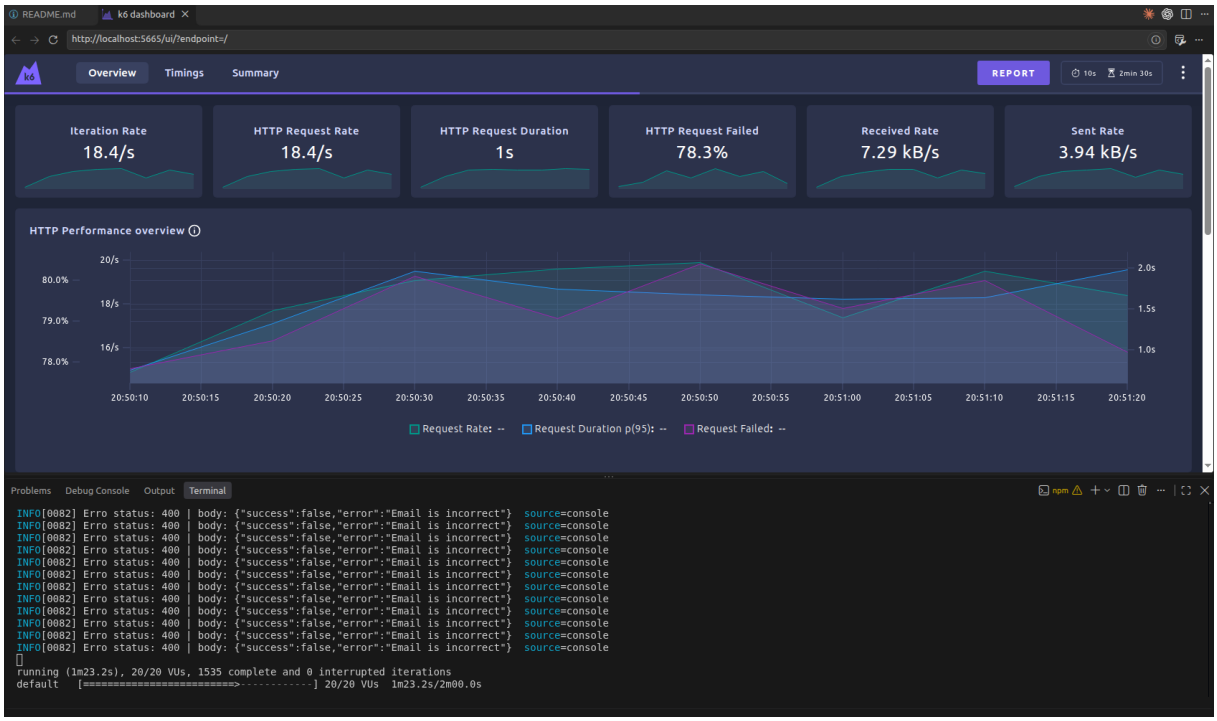
4.7 Testes de performance

Os testes de performance foram implementados com K6 para dois casos de teste: login (POST /public/authUser) e cadastro de usuário (POST /public/addUser). Cada um possui scripts de carga (load), pico (spike) e estresse (stress). Os scripts de carga seguem a configuração exigida:

Parâmetro	Valor
Usuários virtuais	20
Tempo de execução	2 minutos
Rampup	20 segundos
Estágios	20 s subindo até 20 VUs + 1 min 40 s mantendo 20 VUs
Massa de dados (login)	user1_ebac a user5_ebac, senha psw!ebac@test, em rodízio
Validações (checks)	Status 200 e mensagem de retorno esperada

Observação. Na última validação do endpoint de login, apenas o user1_ebac autenticou; user2_ebac a user5_ebac retornaram erro 400 (“Email is incorrect”), o que faz os checks de login falharem para esses usuários.

Anexo 12 – Execução em tempo real dos testes



Autor: Autoral

5. CONCLUSÃO

A realização deste trabalho me permitiu percorrer, de ponta a ponta, o fluxo de um Quality Engineer: partir das histórias de usuário e de suas regras de negócio, escrever critérios de aceitação em Gherkin, projetar 49 casos de teste com técnicas como partição de equivalência, valor limite e tabela de decisão, automatizar os casos de maior valor e executar tudo em integração contínua, com relatórios como evidência.

Uma das principais lições foi a importância de ligar cada caso de teste a uma regra de negócio. Ao comparar os critérios que eu havia escrito com as regras das histórias, percebi lacunas e precisei ajustá-los, o que mostrou que a rastreabilidade entre regra, critério e teste evita retrabalho.

Na automação de interface, trabalhar com Cypress e Playwright, ambos com Page Object e uma camada de fluxo, deixou claras as diferenças de produtividade, depuração e integração entre as ferramentas, e a minha preferência para dar continuidade ao projeto é o Cypress. Na performance, o K6 expôs um problema de massa de dados: apenas um dos cinco usuários definidos para o login autenticava no ambiente. Isso reforçou que o resultado de um teste de carga só é confiável quando os dados e o ambiente estão sob controle.

Na integração contínua, aprendi a usar GitHub Actions com secrets, cache de dependências e artefatos, e a organizar os jobs para que uma falha não impeça a geração dos demais relatórios. Levo desta experiência uma forma de trabalhar mais organizada e orientada a evidências, que pretendo aplicar no meu dia a dia profissional.

6. REFERÊNCIAS BIBLIOGRÁFICAS

AJV. **Ajv JSON schema validator**. Disponível em: https://ajv.js.org/. Acesso em: 1 nov. 2026.

ALLURE. **Allure Report documentation**. Disponível em: https://allurereport.org/docs/. Acesso em: 1 nov. 2026.

CUCUMBER. **Gherkin reference**. Disponível em: https://cucumber.io/docs/gherkin/reference/. Acesso em: 1 nov. 2026.

CYPRESS. **Cypress documentation**. Disponível em: https://docs.cypress.io/. Acesso em: 1 nov. 2026.

ESCOLA BRITÂNICA DE ARTES CRIATIVAS E TECNOLOGIA (EBAC). **Curso Profissão: Engenheiro de Qualidade de Software**. São Paulo: EBAC, 2026.

GITHUB. **GitHub Actions documentation**. Disponível em: https://docs.github.com/en/actions. Acesso em: 1 nov. 2026.

GRAFANA LABS. **Grafana k6 documentation**. Disponível em: https://grafana.com/docs/k6/latest/. Acesso em: 1 nov. 2026.

JEST. **Jest documentation**. Disponível em: https://jestjs.io/docs/getting-started. Acesso em: 1 nov. 2026.

MICROSOFT. **Playwright for Python documentation**. Disponível em: https://playwright.dev/python/. Acesso em: 1 nov. 2026.

PYTEST. **pytest documentation**. Disponível em: https://docs.pytest.org/. Acesso em: 1 nov. 2026.

SUPERTEST. **SuperTest: HTTP assertions made easy via SuperAgent**. Disponível em: https://github.com/ladjs/supertest. Acesso em: 1 nov. 2026.

WOOCOMMERCE. **WooCommerce REST API documentation**. Disponível em: https://woocommerce.github.io/woocommerce-rest-api-docs/. Acesso em: 1 nov. 2026.